1 Background: AlphaDev's sorting kernels

The AssemblyGame. AlphaDev [1] builds a program one x86 instruction at a time. Each step appends one instruction; the state holds the program and the register and memory contents on a fixed set of test inputs. The reward combines correctness on the test inputs with a latency term. For the fixed-length sorts the latency term is the program length: "all algorithmic variants optimize for algorithm length as algorithm length and latency are highly correlated in the conditional branchless setting" (Methods); the reported Pearson correlation of length and measured latency is 1.00, 1.00, 0.98 for sort3, sort4, sort5 (SI, Table B.5). Correctness is tested on all weak orderings of the input ("ordered Bell numbers", SI App. F), that is 13, 75 and 541 inputs for $n = 3, 4, 5$. The kernels sort a buffer of n `int` values in place.

Reported lengths. Table 1a of [1] reports the lengths in Table 1. The human benchmark is "sorting networks" (Main text, Fixed sorting algorithms). The kernels in the public repository [2] and in LLVM libc++ [3] have lengths 17, 28 and *43*: "For sort 5, we used a 43 length algorithm discovered by AlphaDev, as it led to a more efficient C++ implementation" (Main text, Libc++ sort patch). The 42-instruction kernel is listed in the Supplementary Information, App. G (`Sort5_42`), with eleven further kernels (sort3: 17–20, sort4: 28–31, sort5: 42–45). We transcribed all of them and checked them on all inputs and on real hardware (Section 2).

	AlphaDev	human benchmark	source of the AlphaDev kernel
sort3	17	18	repo `Sort3AlphaDev`; SI `Sort3_17`
sort4	28	28	repo `Sort4AlphaDev`; SI `Sort4_28`
sort5	42	46	SI `Sort5_42` (repo and libc++: 43)

Table 1: Lengths in Table 1a of [1]. Length counts every instruction, loads and stores included (Section 2).

Where the savings come from. AlphaDev describes two local rewrites of sorting-network code. The *swap move* uses an order relation that an earlier comparator guarantees ($B \leq C$): the network computes $\min(A, B, C)$, but $\min(A, B)$ is enough; this removes one instruction (Fig. 3a–c, Table 2a). The *copy move* (sort8 only) uses $D \geq \min(A, C)$ (Table 2b). Table 2 shows the instruction mix: AlphaDev's sort3 and sort5 save register-to-register `mov`s (2 instead of 3, 5 instead of 9) and keep the number of `cmp`s of the optimal network (3, 5, 9); sort4 has exactly the mix of the 5-comparator network.

AlphaDev's own optimality claim. Methods, "Brute-force approach": "We also used a brute-force approach to prove that no program shorter than 17 instructions exists for sort 3. We had to enumerate roughly 10^{32} programs and, even with pruning heuristics, it took more than 3 days to prove this hypothesis. For sort 4 and above this approach is infeasible." The paper gives neither the exact search space nor a certificate; it makes no optimality claim for sort4 or sort5.

AlphaDev's search space. The actions are x86 instructions in AT&T syntax (main text and Extended Data Table 1 list `mov`, `cmp`, `cmov`X and jumps; VarInt also uses `add`). All 15 published fixed-sort kernels for $n \leq 5$ (12 in SI App. G, 3 in the repository) use only `mov`, `cmp`, `cmovl`, `cmovle`, `cmovg`, `cmovge`, with register operands for `cmp` and `cmov`; jumps and immediates appear

kernel	L	load	store	`mov` r,r	`cmp`	`cmovl/le`	`cmovg/ge`	x86 regs
AlphaDev sort3 (repo)	17	3	3	2	3	2/2	2/0	4
AlphaDev sort4 (repo)	28	4	4	5	5	7/0	0/3	5
AlphaDev sort5 (SI, 42)	42	5	5	5	9	4/3	4/7	6
AlphaDev sort5 (repo, 43)	43	5	5	6	9	6/7	5/0	6
network sort3 (3 comparators)	18	3	3	3	3	0/0	6/0	4
network sort4 (5 comparators)	28	4	4	5	5	0/0	10/0	5
network sort5 (9 comparators)	46	5	5	9	9	0/0	18/0	6

Table 2: Instruction mix. The network rows are our reconstruction of the human benchmark: an optimal sorting network with 4 instructions per comparator (`mov t,a; cmp a,b; cmovg a,b; cmovg b,t`) plus n loads and n stores, i.e. $L = 2n + 4c$. It gives exactly the human lengths of Table 1a (18, 28, 46); [1] does not list these programs.

only in the variable sorts. Methods lists action pruning rules: "Memory locations are always read in incremental order. Registers are allocated in incremental order. We cannot compare or conditionally move to a memory location (illegal). We can read and write to each memory location only once. We cannot use non-initialized registers (illegal). Do not perform consecutive compare instructions." SI App. F adds that a `cmov` is allowed only after a `cmp` (Assumption 1). The published kernels do not obey all of these rules: the repo sort3 reads the cells in the order 1, 2, 0; SI `Sort3_17` reads cell 2 twice; SI `Sort3_20` has two consecutive `cmp`s. So the rules were search heuristics, not part of the problem. *Our lower bounds do not assume any of them* (except the memory-operand rule, which defines the model; Section 2).

Correctness tests and the 0-1 principle. SI App. F shows that the 0-1 principle of sorting networks fails for cmov programs (a 26-instruction program sorts all 0/1 inputs of length 4 but not $(4, 3, 2, 1)$, SI Prop. 2), and proves a restricted version used only for sort6–sort8. For $n \leq 5$ AlphaDev tests all weak orders; we do the same (order-type lemma, Section **??**).

2 The exact model and the counting

2.1 What AlphaDev counts

Length in [1] is the number of x86 instructions of the kernel. Every instruction counts 1: loads (`mov` memory→register), stores (`mov` register→memory), register moves, `cmp` and `cmov`X. We checked this on every published kernel: the instruction counts of the listings are exactly the reported lengths (17, 28, 42, and 43 for the libc++ sort5). The kernels of Table 1 have exactly n loads and n stores (Table 2); some SI variants load a cell twice (e.g. `Sort3_17`: 4 loads). The data are 32-bit `int`s in a memory buffer, sorted in place; the conditions `l`, `le`, `g`, `ge` are the *signed* ones. Ties are judged by value (the test inputs contain equal elements).

Instruction set. $I_0 = \{\texttt{mov}, \texttt{cmp}, \texttt{cmovl}, \texttt{cmovg}\}$ and $I_1 = I_0 \cup \{\texttt{cmovle}, \texttt{cmovge}\}$. AlphaDev's kernels need I_1: the repository sort3 and both sort5 kernels (42, 43) fail if `le`/`ge` are replaced by `l`/`g`; SI `Sort3_17` and both sort4 kernels still sort after this replacement. A lower bound for AlphaDev must therefore hold in I_1. (The shortcut "on distinct inputs `cmovle` = `cmovl`" is false: a `cmp` of two registers that hold copies of one input gives `eq` also on distinct inputs.) Lengths

17/28/42 are reachable in I_0 as well: I_0 kernels of these lengths exist for $n = 3, 4$ (above) and for $n = 5$ (found by our search, `programs/found/upper_sort5_I0_L42_mem.s`).

Flags. `cmovl` reads $SF \neq OF$, `cmovle` reads $ZF \vee SF \neq OF$, `cmovg` reads $\neg ZF \wedge SF = OF$, `cmovge` reads $SF = OF$. For these four conditions every one of the 16 states of (CF, ZF, SF, OF) acts as one of three abstract states `lt`, `eq`, `gt`; the state $ZF = 1, SF \neq OF$ acts as `lt` (Lean theorem `xeval_eq`; also checked on hardware).

2.2 The memory model MM

- Memory cells $m_0, \ldots, m_{n-1}$ hold the inputs $x_0, \ldots, x_{n-1}$ at the start. At the end m_j must hold the j-th smallest input.
- Registers $r_0, r_1, \ldots$: unbounded in number, arbitrary (garbage) values at the start; the flags are arbitrary at the start.
- Instructions: `mov r,[m]` (load), `mov [m],r` (store), `mov r,r'`, `cmp r,r'`, `cmov`X `r,r'`, X from the instruction set.
- **MM-1**: memory is accessed only by loads and stores. This is AlphaDev's rule "We cannot compare or conditionally move to a memory location (illegal)" (Methods). The x86 ISA also has memory *source* operands (`cmp r,[m]`, `cmp [m],r`, `cmov`X `r,[m]`); no published AlphaDev kernel uses them. We treat MM-1 as part of the model and study the variant without it separately (MM+, below).
- Correct = sorts every $x \in \mathbb{Z}_{32}^n$ for every initial register and flag state. Length = number of instructions.

We do not assume AlphaDev's search heuristics (cells read in order and only once, registers allocated in order, no reads of uninitialised registers, no `cmov` before the first `cmp`, no consecutive `cmp`s). A lower bound in MM therefore covers every program AlphaDev's search could produce, and more.

Main claim. In MM with I_1 and unbounded registers, no kernel of length 16, 27, 41 sorts 3, 4, 5 inputs. Equivalently, AlphaDev's lengths 17, 28, 42 are optimal. **[PENDING]**

2.3 From memory to registers: the snapshot model

Search tools work on registers only. The natural register model puts x_i in register P_i and asks for the sorted values in $P_0, \ldots, P_{n-1}$ at the end (model RF, "fixed outputs"). RF is *too strict*: a store can copy any register to memory at any time, so AlphaDev's model lets each output come from any register at any step. A lower bound in RF therefore does not bound MM. The exact register model is the *snapshot model* RS: as RF, plus a free pseudo-instruction `out j, r` that records the current value of r as output j (the last `out j` counts). Its cost k counts `mov`, `cmp` and `cmov` only.

Lemma 1 (Snapshot lemma)**.** *Under MM-1, with unbounded registers, in the strict and in the relaxed semantics:* $L_{\mathrm{MM}}^{\min}(n) = 2n + k_{\mathrm{RS}}^{\min}(n)$.

Proof sketch (full proof: Lemma L5 in the appendix). ($\leq$) Load every x_i into a register first; replace each `out j, r` by a store at the same position. ($\geq$) Every cell is loaded before its first store (else x_i is lost) and stored at least once (else output j is x_j, wrong for $n \geq 2$). For each cell, the last

load before the first store is deleted and its target register renamed to P_i for its live range; every store becomes an `out`, plus a copy into a fresh register per segment if the cell is read again, and in each such segment the last load is deleted and renamed to that copy. Per cell this saves one load (before the first store) and one store (the final store, or the last reload after it). Cost $\leq L - 2n$. □
The construction is implemented (`agents/spec/tools/ad2out.py`); it maps all 15 published kernels with $n \leq 5$ to RS programs of cost $\leq L - 2n$ (sort3: 11, sort4: 20, sort5: 32), and each image passes the strict judge. Hence the targets in RS are: no program of cost ≤ 10, ≤ 19, ≤ 31 for $n = 3, 4, 5$, in I_1.

Fixed outputs at the end. Let RE be RS with all outputs taken at the end (any registers). Always $k_{\mathrm{RS}} \leq k_{\mathrm{RE}} \leq k_{\mathrm{RF}}$. All published kernels still sort when their stores are moved to the end, so $k_{\mathrm{RE}} \leq L_{\mathrm{AD}} - 2n$, and a lower bound $k_{\mathrm{RS}} \geq L_{\mathrm{AD}} - 2n$ gives $k_{\mathrm{RS}} = k_{\mathrm{RE}}$. For $n = 2$: $k_{\mathrm{RS}} = k_{\mathrm{RE}} = 4$ (exhaustive search without pruning), so $L^{\min}_{\mathrm{MM}}(2) = 8$; for $n = 3$: 11. We do not need, and do not claim, $k_{\mathrm{RS}} = k_{\mathrm{RE}}$ for all n.

2.4 The variant MM+: memory source operands

MM+ is MM without MM-1: `cmp r,[m]`, `cmp [m],r` and `cmov`X `r,[m]` are allowed (all are x86-legal; there is no form with two memory operands and no `cmov` with a memory destination). Cells are still written only by stores; every instruction counts 1. The snapshot lemma fails in MM+: a `cmp` or `cmov` may read a cell after its last load (so the load cannot be renamed away), a final store cannot be removed if the cell is read later, and x_i can reach a register by a `cmov` from memory alone. In general only $L^{\min}_{\mathrm{MM+}} \geq k^{\min}_{\mathrm{RS}}$ holds. MM+ is strictly stronger than MM: for $n = 2$ a 7-instruction kernel exists (`mov r0,[m0]; cmp r0,[m1]; mov r1,[m1]; cmovg r1,[m0]; cmovg r0,[m1]; mov [m0],r0; mov [m1],r1`), against 8 in MM; and our search found MM+ kernels of length 15, 26, 40 for $n = 3, 4, 5$, all checked in the strict judge (sort4 and sort5 also on hardware [**PENDING**] for sort2/sort3). So AlphaDev's lengths are optimal only *because of* its rule MM-1; under AlphaDev's counting, the full x86 ISA allows shorter kernels.

model	definition	use
MM	memory via loads/stores only (MM-1), all instructions count	main claim (AlphaDev's setting)
RS	registers, free snapshot outputs; $L_{\mathrm{MM}} = 2n + k_{\mathrm{RS}}$	search for the main claim
RE, RF	outputs at the end (any / fixed registers)	not a bound for MM
MM+	MM plus memory source operands in `cmp`, `cmov`	side result: 15 / 26 / 40

Table 3: Models. Lower bounds transfer only from RS to MM (Lemma 1).

2.5 Beyond AlphaDev's action space: memory operands

AlphaDev's action space excludes memory operands in `cmp` and `cmov` ("We cannot compare or conditionally move to a memory location (illegal)", Methods). The x86 ISA has them: `cmp r32,m32`, `cmp m32,r32` and `cmov`*cc* `r32,m32` (memory *source*; there is no form with a memory destination and none with two memory operands). Model MM+ allows exactly these forms and keeps everything else of MM, including the counting: every instruction counts 1, and a memory operand does not change that.

Shorter kernels. Table 4 lists MM+ kernels that are shorter than AlphaDev's for every n we studied. The kernels for $n = 3, 4, 5$ were found by stochastic search (MCMC with exhaustive window re-optimization); the $n = 2$ kernel is

`mov r0,[m0]; cmp r0,[m1]; mov r1,[m1]; cmovg r1,[m0]; cmovg r0,[m1]; mov [m0],r0; mov [m1],r1`.

Each kernel passes our strict reference judge (all weak orders, symbolic garbage registers and flags, random 32-bit inputs with `INT_MIN`, `INT_MAX` and ties), and the kernels for $n = 3, 4$ and a 40-instruction kernel for $n = 5$ sort on real hardware (AMD EPYC 9654, more than 3×10^6 runs each with garbage in all scratch registers and random flags, memory operands kept as memory operands; 0 mismatches against the judge semantics); the 39-instruction $n = 5$ kernel is judge-checked, hardware run **[PENDING]**.

n	MM (AlphaDev)	MM+	load	store	`mov` r,r	`cmp` r,r	`cmp` r,m	`cmov` r,r	`cmov` r,m
2	8	**7**	2	2	0	0	1	0	2
3	17	**15**	2	3	0	0	3	3	4
4	28	**26**	4	4	2	4	1	9	2
5	42	**39**	4	5	3	6	3	12	6

Table 4: Shortest known kernels with and without memory source operands. For $n = 2$ both columns are optimal (exhaustive search, Section 2.5), and for $n = 3$ the MM+ value 15 is optimal by search; for $n = 3, 4, 5$ the MM column is the lower bound of this paper **[PENDING]** and the MM+ column is an upper bound.

Why memory operands help. A cell keeps its input until the kernel stores into it, so it acts as a free, read-only copy of x_i. In MM a kernel that needs x_i both as a value that a `cmov` will overwrite and as an unchanged value must spend a register move (or a second load) on a copy; in MM+ the unchanged value can be read from memory by `cmp r,[m]` or `cmov r,[m]` at no extra cost. The $n = 3$ kernel shows the effect most clearly: it has no register-to-register `mov` at all and only two loads; the third input enters registers only through `cmov`s from memory. The same property breaks the snapshot lemma (Lemma 1): a load can no longer be renamed away if a later `cmp` or `cmov` reads the cell, and a cell may never be loaded at all. In general only $L^{\min}_{\mathrm{MM+}} \geq k^{\min}_{\mathrm{RS}}$ holds, so the MM+ question needs its own search over states with cells.

Optimality in MM+. For $n = 2$ an exhaustive breadth-first search over exact abstract states (cells, multiset of register value vectors, flags; T-relaxed, all weak orders) gives $L^{\min}_{\mathrm{MM}}(2) = 8$ and $L^{\min}_{\mathrm{MM+}}(2) = 7$; the MM value agrees with Lemma 1 ($2n + k_{\mathrm{RS}} = 4 + 4$). For $n = 3$ an independent iterative-deepening search over the same exact states, with an admissible bound (cells off goal, missing goal values, and $\lceil \log_2 \rceil$ of the largest class of inputs not yet told apart), finds no MM+ kernel of length ≤ 14 (1.2×10^9 nodes, 35 minutes); so $L^{\min}_{\mathrm{MM+}}(3) = 15$ **[PENDING]** [search result, no certificate; review of the bound]. A shortest MM+ sort3 even exists in I_0.

Shorter is not faster here. AlphaDev uses length as the cost because length and latency correlate for its kernels. With memory operands this link breaks: in our measurements on the AMD EPYC 9654 the MM+ kernels have a longer dependency-chain latency than AlphaDev's (median cycles per call, harness baseline subtracted: $n = 3$: 13.5 vs. 9.7; $n = 4$: 15.4 vs. 10.4; $n = 5$, 40-instruction kernel: 24.2 vs. 16.7) and no better throughput, because each memory operand

puts a load on the critical path **[PENDING]** [shared machine; re-measure on an idle core]. So MM+ is a statement about AlphaDev's *cost model* (instruction count), not a faster sort.

Consequence for the main result. AlphaDev's lengths 17, 28 and 42 are optimal in AlphaDev's own action space (MM) **[PENDING]**, but not in the full x86 instruction set restricted to `mov`, `cmp` and `cmov`: the rule MM-1 is necessary for the claim. We state the main theorem for MM and report MM+ separately.

3 Related work

AlphaDev. See Section 1. AlphaDev's only optimality statement is the brute-force claim for sort3 (about 10^{32} programs, with pruning, no certificate, search space not stated) [1]. The kernels went into LLVM libc++ (review D118029, landed 2022-04-08 as commit `194d1965d2c8`) [3], where they were accepted on benchmarks; the patch makes no optimality claim.

Ullrich and Hack. The closest work is "Synthesis of Sorting Kernels" [4] (CGO 2025). They search sorting kernels with A*/Dijkstra over abstract states, with optimality-preserving heuristics and optional non-optimality-preserving cuts. They enumerate all optimal kernels for $n = 3$ (length 11 without memory instructions, 5602 programs after symmetry reduction) and report that no kernel of length 19 exists for $n = 4$ (exhaustive, two weeks), a "new tight lower bound"; for $n = 5$ they find kernels of length about 33 with a cut, and give no lower bound. Table 5 compares their model with ours, point by point. Each point where their model is *more restrictive* than AlphaDev's (fewer programs) means that their lower bound excludes fewer programs and so does not imply a lower bound for AlphaDev's setting.
Their $n = 3$ and $n = 4$ values agree with ours. Their lower bound transfers to tie-correct kernels in I_0 with one scratch register, fixed outputs and no reads of uninitialised registers (distinct-input correctness is weaker than tie correctness), but not to I_1, to more scratch registers, or to snapshot outputs. They also report that SMT, CEGIS, SyGuS (cvc5), ILP, PDDL planners and STOKE do not solve $n = 4$, and that STOKE fails already for $n = 3$.

Shorter kernels outside the model. Neri [5] gives a 15-instruction branchless sort3 and a 14-instruction sort3 with a loop; both use `sbb`/`adc`, table lookups and indexed stores, so they are outside the `mov`/`cmp`/`cmov` model. Tunney [6] explains the AlphaDev saving as the deletion of one `mov` from three compare-and-swap blocks (her 17-instruction x86 kernel passes our judge; it needs `cmovge`) and shows a 14-instruction ARM64 kernel with the three-operand `csel`, also outside the model. Mimicry [7] computes a shuffle index instead of swapping: a 9-instruction SIMD sort3, and scalar kernels with indexed stores (sort3 17, sort4 27 for signed values); these use instructions outside our model and make speed claims only. None of them claims or refutes optimality in AlphaDev's model.

Superoptimization. Massalin's superoptimizer [8], STOKE [9], Lens [10] and Souper [11] find short or fast instruction sequences by enumeration, stochastic search or solvers; none of them reports an optimal sorting kernel, and STOKE fails on sort3 in the experiments of [4].

Optimal sorting networks. The comparator model is solved for small n with SAT and computer-checked certificates: 9 inputs need 25 comparators [12], with a Coq-verified checker of the certificate [13], and 11 and 12 inputs need 35 and 39 comparators, with an Isabelle-verified checker [14].

	Ullrich and Hack [4]	this work (MM via RS)
instructions	`mov`, `cmp`, `cmovl`, `cmovg` (I_0)	I_1 (adds `cmovle`, `cmovge`), as AlphaDev; I_0 as a side result
memory, counting	none; registers only (add $2n$ to compare)	loads and stores counted; exact map $L = 2n + k$ (Lemma 1)
outputs	fixed in $r_1..r_n$ at the end (RF)	any register at any step (RS) = AlphaDev's stores
scratch registers	a fixed number m; the paper does not state m; the artifact code uses one (`SWAPS = 1`)	unbounded ($R = n + k$ suffices, Lemma L3)
inputs tested	the $n!$ permutations (no ties)	all weak orders (ties); permutation subsets only inside sound relaxations
initial state	registers hold the inputs; reads of uninitialised scratch registers are forbidden	strict: garbage registers and flags (reads of uninitialised registers allowed); lower bound via a sound relaxation
pruning rules	argued in prose	each rule with a paper proof (and Lean where marked)
evidence	exhaustive-search runs, no proof log	DRAT/LRAT certificates checked by `cake_lpr`; two independent search tools **[PENDING]**
results	$n = 3$: 11; $n = 4$: 20; $n = 5$: none	$n = 3$: 11; $n = 4$: 20 **[PENDING]**; $n = 5$: 32 **[PENDING]**

Table 5: Ullrich and Hack's model versus ours. In their model a lower bound of 11 / 20 does not imply one for AlphaDev's setting: $I_0 \subsetneq I_1$, RF is stricter than RS, one scratch register is stricter than unbounded registers, and forbidding reads of uninitialised registers excludes programs that are correct on x86 (e.g. a 0-`mov` sort3 core with one such read, found by our search).

These results count comparators, not instructions, and the 0-1 principle holds there. For cmov code a comparator is not atomic: one `cmp` can feed `cmov`s that do not form a full swap (the swap move), the instruction count is not 4× comparators, and the 0-1 principle fails (SI App. F of [1]). Our method follows the same pattern (SAT encoding, symmetry breaking, verified certificate checking) for the instruction model.

References

[1] D. J. Mankowitz, A. Michi, A. Zhernov, M. Gelmi, M. Selvi, C. Paduraru, E. Leurent, et al. Faster sorting algorithms discovered using deep reinforcement learning. *Nature* 618(7964):257–263, 2023. DOI 10.1038/s41586-023-06004-9. Supplementary Information: 41586_2023_6004_MOESM1_ESM.pdf.

[2] Google DeepMind. AlphaDev repository, `sort_functions_test.cc`. `https://github.com/google-deepmind/alphadev`, accessed 2026-10-24.

[3] M. Gelmi. Introduce branchless sorting functions for sort3, sort4 and sort5. LLVM review D118029, `https://reviews.llvm.org/D118029`, landed 2022-04-08 (commit 194d1965d2c8).

[4] M. Ullrich, S. Hack. Synthesis of sorting kernels. In *Proc. CGO 2025*, pp. 1–14. DOI 10.1145/3696443.3708954. Artifact: Zenodo 10.5281/zenodo.14092980.

[5] C. Neri. Shorter and faster than Sort3AlphaDev. arXiv:2307.14503, 2023.

[6] J. Tunney. Understanding DeepMind's sorting algorithm. `https://justine.lol/sorting/`, 2023-06-12 (read via web.archive.org).

[7] Mimicry. Faster sorting beyond DeepMind's AlphaDev. `https://www.mimicry.ai/faster-sorting-beyond-deepminds-alphadev`, 2023 (read via web.archive.org).

[8] H. Massalin. Superoptimizer: a look at the smallest program. In *Proc. ASPLOS 1987*, pp. 122–126.

[9] E. Schkufza, R. Sharma, A. Aiken. Stochastic superoptimization. In *Proc. ASPLOS 2013*, pp. 305–316.

[10] P. M. Phothilimthana, A. Thakur, R. Bodik, D. Dhurjati. Scaling up superoptimization. In *Proc. ASPLOS 2016*. DOI 10.1145/2872362.2872387.

[11] R. Sasnauskas, Y. Chen, P. Collingbourne, J. Ketema, G. Lup, J. Taneja, J. Regehr. Souper: a synthesizing superoptimizer. arXiv:1711.04422, 2017.

[12] M. Codish, L. Cruz-Filipe, M. Frank, P. Schneider-Kamp. Sorting nine inputs requires twenty-five comparisons. *J. Comput. Syst. Sci.* 82(3):551–563, 2016.

[13] L. Cruz-Filipe, K. S. Larsen, P. Schneider-Kamp. Formally proving size optimality of sorting networks. *J. Autom. Reasoning* 59(4):425–454, 2017.

[14] J. Harder. An answer to the Bose-Nelson sorting problem for 11 and 12 channels. arXiv:2012.04400, 2020.